Exchange Core Matching Engine

Feature specification — production matching engine, persistence, connectivity, compliance, and operations.

17	307	9	6	5	12
Packages	Tests	WAL events	Order types	Session phases	Metric types

1. Core Matching Engine

- Price-time priority (FIFO) matching with red-black tree price levels — `internal/orderbook/`
- Fixed-point int64 prices with 1e8 scaling — `internal/models/`
- Single goroutine per instrument — lock-free, single-writer architecture — `internal/engine/`
- Self-trade prevention (STP): `CANCEL_RESTING`, `CANCEL_INCOMING`, `CANCEL_BOTH`
- Order amendment: in-place qty decrease preserves priority; cancel + re-insert on price change
- Mass cancel with filters by instrument / client / side
- Stop order activation with cascade (capped at 100 iterations)

2. Order Types

LIMIT	MARKET	IOC
FOK	STOP (stop-market)	STOP_LIMIT

3. Persistence & Recovery

internal/wal

- Binary WAL with CRC32 integrity, per-instrument files
- Configurable fsync mode: `fsync`, `fdatasync`, `none`
- 9 event types: OrderAdd, OrderCancel, StopActivation, OrderAmend, MassCancel, SessionTransition, AuctionUncross, OrderRejected, TradeExecuted
- Periodic snapshots with atomic rename + old-WAL cleanup
- Full crash recovery via snapshot + WAL replay

4. Security & Access Control

- TLS 1.2+ with cert/key paths — `cmd/server`
- API key authentication + HMAC-SHA256 signature verification — `internal/auth/`
- 30-second timestamp window for anti-replay protection
- Per-key permissions (trade, read)
- Token-bucket rate limiting, separate read/write per client — `internal/ratelimit/`

5. Trading Sessions & Auctions

- 5-phase lifecycle: PRE_OPEN → AUCTION_OPEN → CONTINUOUS → AUCTION_CLOSE → CLOSED — `internal/session/`
- Opening / closing auctions with equilibrium price (max matched volume) — `internal/auction/`
- Uncross: all fills at the single equilibrium price (market orders first)
- Indicative price publication over WebSocket during auctions
- Circuit-breaker halt overrides any phase

6. Safety & Circuit Breakers

internal/circuit

- Price-band checks
- Velocity monitoring
- Fat-finger protection
- Auto-resume with configurable pre-open
- Mass cancel always available during halt (kill switch)

7. Real-Time Data

- WebSocket feed: subscribe per instrument + channel (trades, book, ticker) — `internal/ws/`
- Per-client 256-buffer with backpressure (slow-client disconnect)
- Prometheus metrics at `/metrics` (12 metric types) — `internal/metrics/`
- JSON stats at `/v1/stats` (backward compatible)

8. Institutional Connectivity

- FIX 4.4 gateway: Logon, Logout, Heartbeat, NewOrderSingle, ExecutionReport — `internal/fix/`
- In-house tag=value parser with checksum validation
- Heartbeat, sequence numbers, ClOrdID mapping
- Primary–replica replication via TCP WAL streaming — `internal/replication/`
- Replica failover via promote endpoint

9. Compliance & Regulatory

- Nanosecond timestamps on all orders & trades (MiFID II)
- `ReceivedAt` captured at API ingress for latency tracking
- Monotonic trade sequence numbers
- Audit exporter (NDJSON / CSV from WAL) — `internal/audit/`
- Order-to-trade ratio (OTR) monitoring with ALERT / REJECT — `internal/compliance/`
- Market surveillance (async): spoofing, layering, wash-trading detectors — `internal/surveillance/`

10. REST API

internal/api

Method	Endpoint	Purpose
POST / DELETE / GET / PUT	`/v1/orders`	Place, cancel, query, amend (incl. mass cancel)
GET	`/v1/orderbook/{instrument}`	Order book snapshot
GET	`/v1/instruments`	List instruments
GET	`/v1/stats, /v1/stats/{instrument}`	Engine + per-instrument stats
GET	`/metrics`	Prometheus metrics
WS	`/v1/ws`	Real-time market data
GET	`/health`	Liveness probe

11. Operational

- All features independently toggleable via env vars or JSON config — `internal/config/`
- Graceful shutdown
- Dockerfile + single-binary build
- Zero mandatory external deps beyond `nhooyr.io/websocket` and `prometheus/client_golang`
- 307 tests across 17 packages

Document generated from the project README and 17 internal packages. This is a feature inventory; for protocol details, refer to the per-package README and source.